

КУРСОВОЙ ПРОЕКТ

Трекер расходов

Веб-приложение для учёта личных финансов

Студент: Моисеенко О.Н., группа 253-325 Преподаватель: Красникова И.Н.

Содержание

01 Введение

Актуальность и проблематика

02 Цель и задачи

Постановка задачи проекта

03 Анализ предметной области

Обзор решений и требования

04 Проектирование БД

ER-диаграмма и логическая модель

05 Реализация

Архитектура, стек, ключевые алгоритмы

06 SQL-запросы

Пять ключевых запросов

07 Тестирование

Функциональное и нефункциональное

08 Заключение

Результаты и выводы

Проблема управления личными финансами

60%

людей не ведут учёт
своих расходов

- Неконтролируемые расходы ведут к финансовой нестабильности
- Люди узнают о дефиците бюджета лишь к концу месяца
- Банковские приложения привязаны к банку и не учитывают наличные
- Онлайн-сервисы требуют регистрации и постоянного интернета
- Табличные решения (Excel) лишены удобства и автоматизации

Цель и задачи проекта

Разработать веб-приложение для учёта личных расходов на Python + Flask + SQLite

Задачи:

- Анализ предметной области и существующих решений
- Проектирование структуры базы данных (ER + логика)
- Реализация таблиц categories, expenses, limits (SQLite)
- Разработка веб-интерфейса на Flask с шаблонами Jinja2
- Реализация аналитических отчётов по месяцам и категориям
- Добавление системы уведомлений о превышении лимитов
- Подготовка документации проекта

Обзор существующих решений

Банковские приложения

Сбербанк, Тинькофф

✓ Автоучёт транзакций

✗ Привязаны к банку, нет наличных

Онлайн-сервисы

Дзен-мани, CoinKeeper

✓ Расширенная аналитика

✗ Нужен интернет + регистрация

Табличные решения

Excel / Google Sheets

✓ Максимальная гибкость

✗ Нет интерфейса, нет автоматизации

Незаполненная ниша: offline-приложение · локальное хранение · без регистрации · умные уведомления

Требования к системе

Функциональные требования

- Ввод расхода: сумма, категория, описание, дата
- Просмотр и удаление транзакций с фильтрацией по месяцу
- Ежемесячный отчёт: разбивка по категориям, история 6 мес.
- Бюджетные лимиты с визуальным прогресс-баром
- Умные уведомления с персонализированными комментариями

Нефункциональные требования



Время отклика UI

< 200 мс



Работа без интернета

Полностью offline



Нет сторонних зависимостей

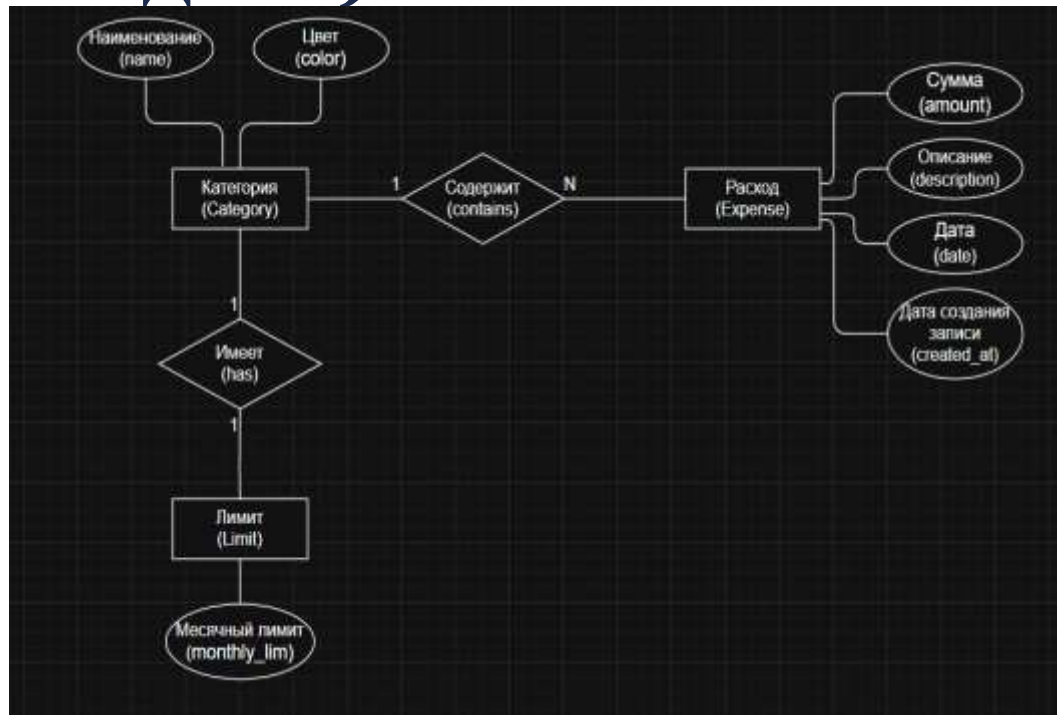
Только stdlib Python



Кросс-платформенность

Win / macOS / Linux

ER-диаграмма (концептуальная модель)



Категория (Category)

Тип расходов: name, color

Расход (Expense)

Транзакция: amount, date, description, created_at

Лимит (Limit)

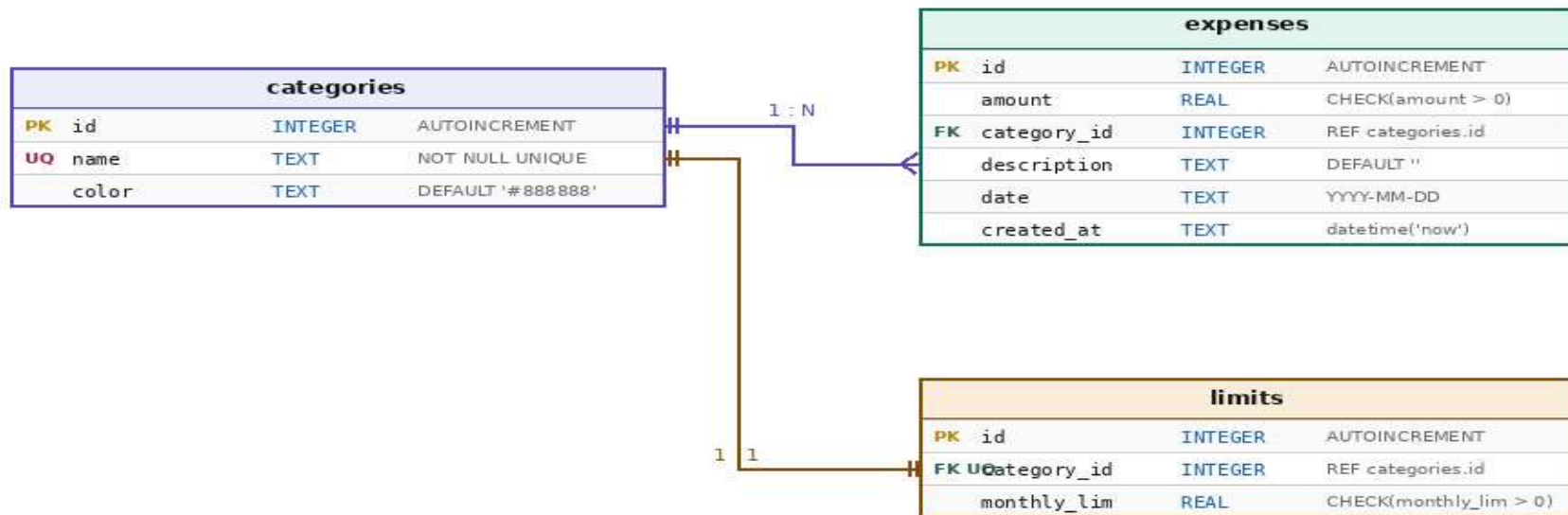
Ограничение: monthly_lim по категории

Связи:

1:N — Категория → Расход

1:1 — Категория → Лимит

Логическая модель данных

**PK** — первичный ключ**FK** — внешний ключ**UQ** — уникальность

Трекер расходов — ER-диаграмма

Логическая модель данных

Таблица categories

id	Автоинкрементный идентификатор
name	Наименование категории (уникальное)
color	Цвет в формате #RRGGBB

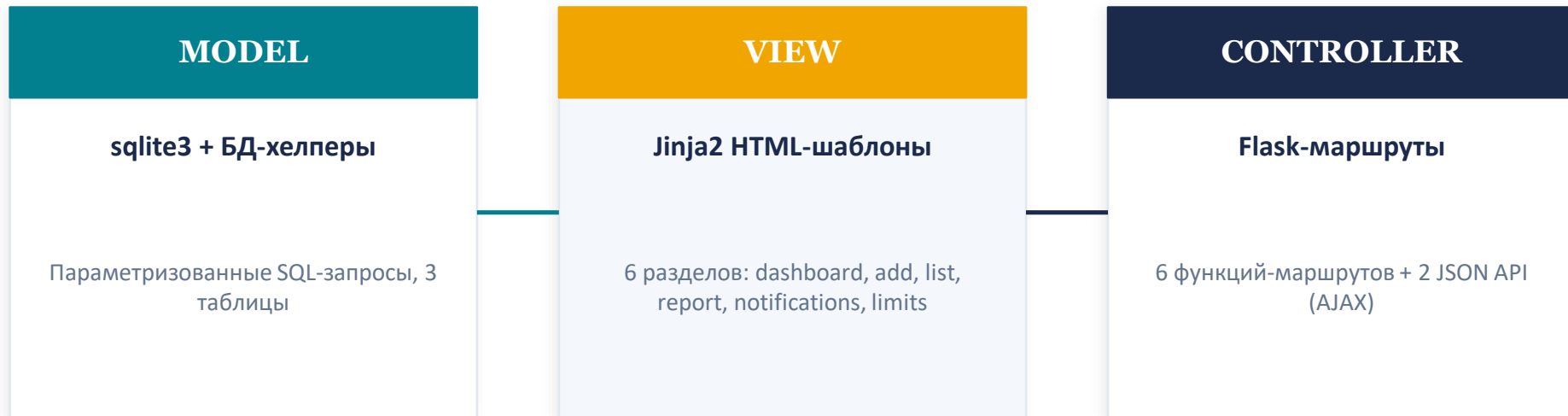
Таблица expenses

id	Автоинкрементный идентификатор
amount	Сумма расхода в рублях
category_id	Ссылка на categories.id
description	Произвольное описание операции
date	Дата расхода, формат YYYY-MM-DD
created_at	Дата-время добавления записи

Таблица limits

id	Автоинкрементный идентификатор
category_id	Ссылка на categories.id
monthly_lim	Месячный бюджетный лимит в рублях

Архитектура и стек технологий



Компонент	Назначение
Python 3.8+	Основной язык программирования
Flask / Jinja2	Веб-фреймворк и шаблонизатор
SQLite / sqlite3	Встроенная реляционная БД
datetime, os, json	Вспомогательные модули stdlib

Разделы веб-приложения



Dashboard

Сводка расходов за месяц, топ категорий



Добавить расход

Форма ввода: сумма, категория, описание, дата



Список расходов

Фильтрация по месяцу, удаление записей



Отчёт

Разбивка по категориям, история 6 месяцев



Уведомления

Умные сообщения при превышении лимитов



Лимиты

Установка бюджетных ограничений с прогресс-баром

Структура модулей app.py

init_db()

Инициализация БД, создание таблиц, вставка демо-данных при первом запуске

report()

Ежемесячный отчёт: разбивка по категориям, история за 6 месяцев

DB Helpers (5 ф-й)

Параметризованные SQL-запросы: get_expenses, get_categories, get_limits...

notifications()

Генерация персонализированных уведомлений при превышении лимитов

dashboard()

Главная страница — сводка расходов за текущий месяц, топ категорий

limits_page()

Просмотр и редактирование бюджетных лимитов по категориям

add() / expenses()

Добавление расхода (POST) и просмотр списка с фильтром по месяцу

api_delete() (2 API)

JSON-эндпоинты для удаления расходов через AJAX без перезагрузки

Ключевые алгоритмы

1

Запрос к БД: расходы по категориям за текущий месяц

2

Сравнение сумм с установленными лимитами

3

Спец. условие: расходы на кофе > 50% лимита?

4

Превышение 100% лимита любой категории?

5

Превышение 80% лимита → предупреждение

Ключевые SQL-запросы

Запрос 1 — Расходы за месяц (JOIN)

```
SELECT e.*, c.name, c.color
FROM expenses e
JOIN categories c ON e.category_id = c.id
WHERE strftime('%Y-%m', e.date) = ?
```

Запрос 2 — Суммы по категориям (GROUP BY)

```
SELECT c.name, SUM(e.amount) AS total
FROM expenses e
JOIN categories c ON e.category_id = c.id
WHERE strftime('%Y-%m', e.date) = ?
GROUP BY c.id
```

Запрос 3 — История за 6 месяцев

```
SELECT strftime('%Y-%m', date) AS month,
       SUM(amount) AS total
FROM expenses
GROUP BY month
ORDER BY month DESC LIMIT 6
```

SQL-запросы: контроль лимитов

Запрос 4 — Превышение лимита на 20% (тройной JOIN)

```
SELECT c.name, SUM(e.amount) AS spent, l.monthly_lim  
FROM expenses e  
JOIN categories c ON e.category_id = c.id  
JOIN limits l ON l.category_id = c.id  
WHERE strftime('%Y-%m', e.date) = ?  
GROUP BY c.id  
HAVING spent > l.monthly_lim * 1.2
```

Запрос 5 — Топ-3 дорогих транзакции

```
SELECT e.amount, e.description, c.name  
FROM expenses e  
JOIN categories c ON e.category_id = c.id  
ORDER BY e.amount DESC  
LIMIT 3
```

Функциональное тестирование

Тест-кейс	Ожидаемый результат	Итог
Добавление расхода с суммой 0	Ошибка валидации	✓
Добавление с некорректной датой	Ошибка валидации	✓
Добавление корректного расхода	Запись сохранена в БД	✓
Удаление расхода	Запись удалена из БД	✓
Смена месяца в фильтре	Данные обновляются	✓
Уведомление при 80%+ лимита	Карточка появляется	✓
Сохранение нового лимита	Запись в таблице limits	✓

Все 7 тест-кейсов пройдены успешно

Нефункциональное тестирование

40–80

МС

Ответ сервера
(дашборд)

150–200

МС

Отображение
в браузере

< 200

МС

Требование по
TechSpec

Кросс-платформенность

Windows

✓ 10 / 11

Ubuntu

✓ 22.04 LTS

macOS

✓ Ventura

Достигнутые результаты

3

таблицы БД

categories, expenses, limits с ограничениями целостности и внешними ключами

6

разделов UI

Адаптивный веб-интерфейс: дашборд, добавление, список, отчёт, уведомления, лимиты

5

SQL-запросов

JOIN, GROUP BY, HAVING, LIMIT, агрегатные функции — полный CRUD

2

JSON API

AJAX-эндпоинты для удаления расходов без перезагрузки страницы

Выводы

- Разработано веб-приложение «Трекер расходов» на Python 3 + Flask + SQLite
- Приложение запускается локально и не требует регистрации в сторонних сервисах
- Реализован полный цикл: проектирование БД → разработка → тестирование
- Реализована система умных персонализированных уведомлений о превышении лимитов
- Приложение успешно протестировано на Windows, Ubuntu и macOS
- Время отклика UI не превысило 200 мс — требование выполнено

Использованные источники

1

Python Software Foundation

Python 3 Documentation — <https://docs.python.org/3/>

2

SQLite

SQLite Documentation — <https://www.sqlite.org/docs.html>

3

Pallets Projects

Flask Documentation — <https://flask.palletsprojects.com/>

Спасибо за внимание!

Трекер расходов — Python + Flask + SQLite

Студент: Моисеенко О.Н., группа 253-325 | **Преподаватель:** Красникова И.Н.